DiceKriging vs RobustGaSP

Gu (2019)

The statistical framework

Prediction

We now provide an example in which the input has one dimension, ranging from [0, 10] (Higdon and others (2002)). Estimation of the range parameters using the `RobustGaSP` package can be done through the following code:

```
library(RobustGaSP)
library(lhs)
set.seed(1)
input <- 10 * maximinLHS(n=15, k=1)
output <- higdon.1.data(input)
model <- rgasp(design = input, response = output)
```

```
## The upper bounds of the range parameters are 395.6186
## The initial values of range parameters are 7.912373
## Start of the optimization  1  :
## The number of iterations is  10
##  The value of the  marginal posterior  function is  2.81014
##  Optimized range parameters are 1.768226
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
## The initial values of range parameters are 0.08251576
## Start of the optimization  2  :
## The number of iterations is  12
##  The value of the  marginal posterior  function is  2.81014
##  Optimized range parameters are 1.768226
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
```

```
model
```

```
##
## Call:
## rgasp(design = input, response = output)
## Mean parameters:  0.01184582
## Variance parameter:  0.5697197
## Range parameters:  1.768226
## Noise parameter:  0
```

The fourth line of the code generates 15 LH samples at $[0, 10]$ through the `maximinLHS` function of the `lhs` package (Carnell (2016)). The function `higdon.1.data` is provided within the `RobustGaSP` package which has the form $y(x) = \sin(2\pi x/10) + 0.2\sin(2\pi x/2.5)$. The third line fits a GaSP model with the robust parameter estimation by marginal posterior modes.

The plot function in `RobustGaSP` package implements the leave-one-out cross validation for a "rgasp" class after the GaSP model is built (see Figure 1 for its output):

```
plot(model)
```

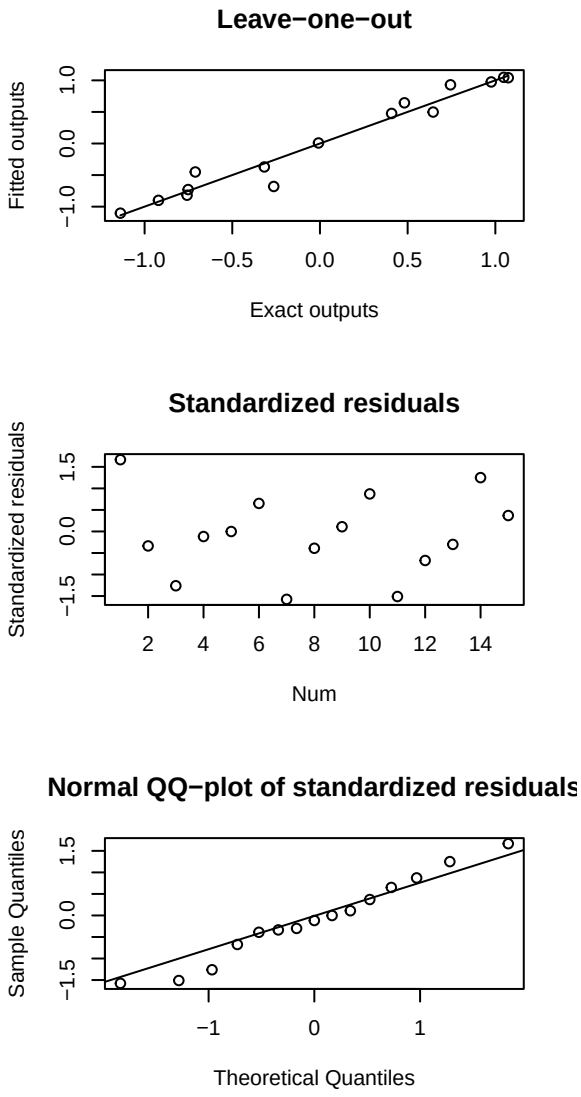


Figure 1: Figure 1

The prediction at a set of input points can be done by the following code:

```
testing_input <- as.matrix(seq(0, 10, 1/50))
model.predict<-predict(model, testing_input)
names(model.predict)
```

```
## [1] "mean"    "lower95" "upper95" "sd"
```

The predict function generates a list containing the predictive mean, lower and upper 95% quantiles and the predictive standard deviation, at each test point $\mathbf{x}^*$. The prediction and the real outputs are plotted in Figure 2, produced by the following code:

```
testing_output <- higdon.1.data(testing_input)
plot(testing_input, model.predict$mean,type='l', col='blue',
     xlab='input', ylab='output')
polygon( c(testing_input,rev(testing_input)), c(model.predict$lower95,
                                                rev(model.predict$upper95)), col = "grey80", border = F
lines(testing_input, testing_output)
lines(testing_input,model.predict$mean, type='l', col='blue')
lines(input, output, type='p')
```

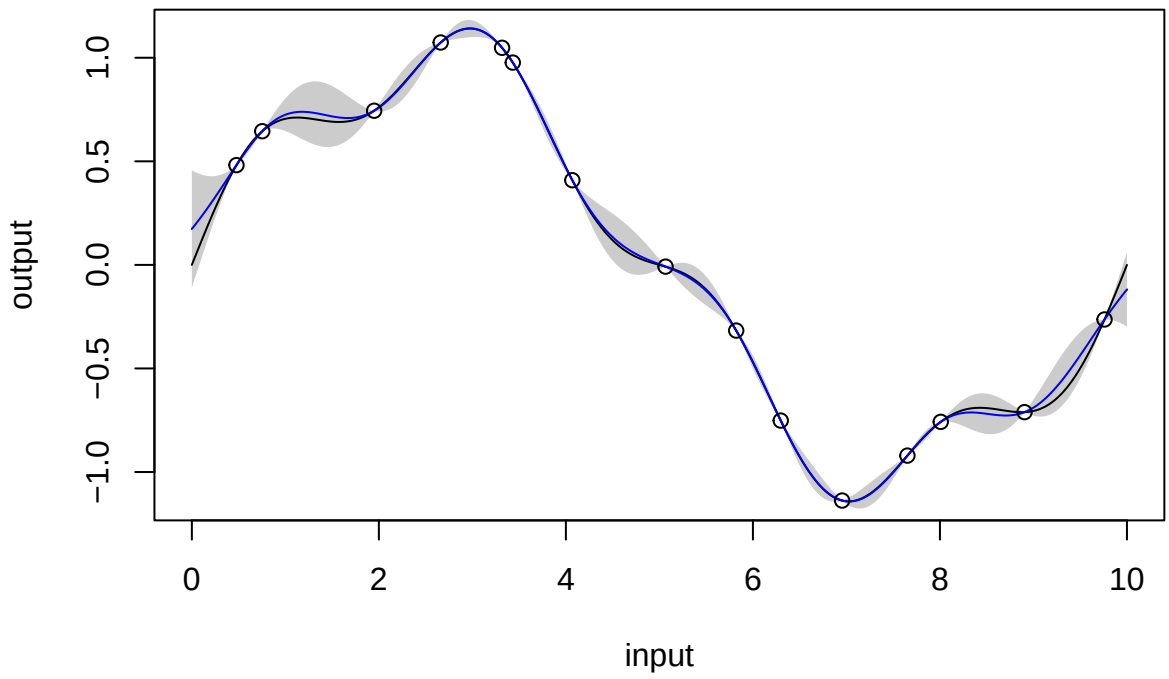


Figure 2: Figure 2

It is also possible to sample from the predictive distribution (which is a multivariate t distribution) using the following code:

```
model.sample <- RobustGaSP::simulate(model, testing_input, num_sample=10)
matplot(testing_input, model.sample, type='l', xlab='input', ylab='output')
lines(input, output,type='p')
```

The plots of 10 posterior predictive samples are shown in Figure 3.

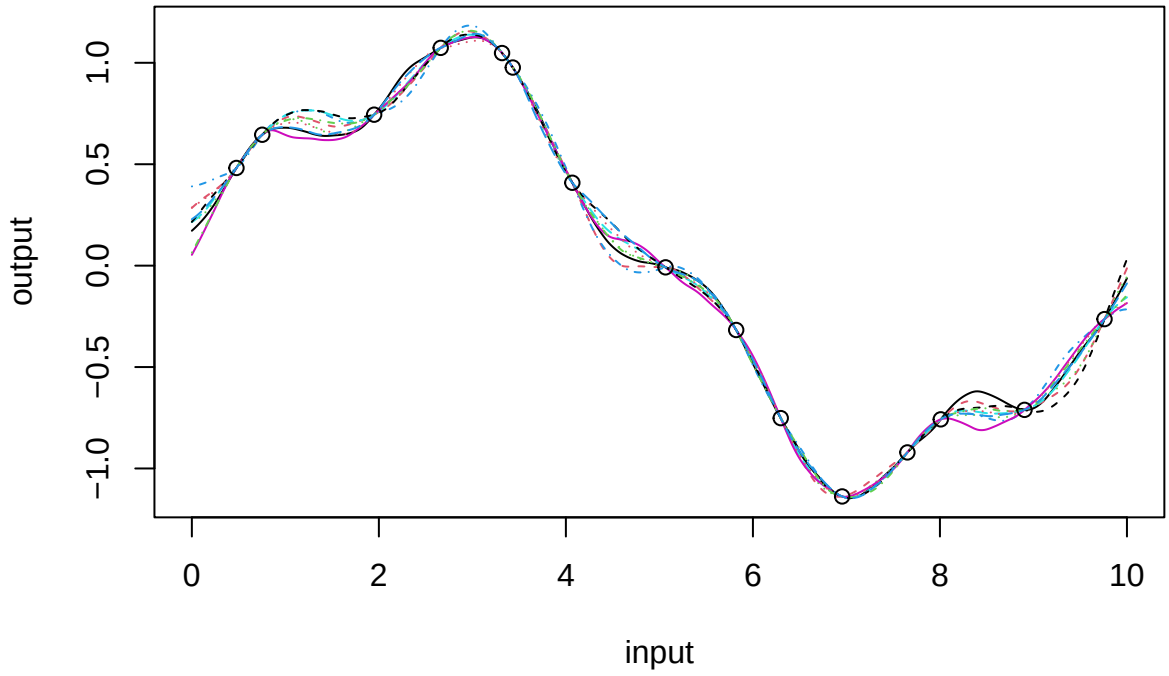


Figure 3: Figure 3

Identification of inert inputs

We use 40 maximin LH samples to find inert inputs at the borehole function through the following code.

```
set.seed(1)
input <- maximinLHS(n=40, k=8) # maximin lhd sample

# rescale the design to the domain of the borehole function
LB <- c(0.05, 100, 63070, 990, 63.1, 700, 1120, 9855)
UB <- c(0.15, 50000, 115600, 1110, 116, 820, 1680, 12045)
range <- UB - LB
for(i in 1:8) {
  input[,i] = LB[i] + range[i] * input[,i]
}
num_obs <- dim(input)[1]
output <- matrix(0,num_obs,1)
for(i in 1:num_obs) {
  output[i] <- borehole(input[i,])
  }
m <- rgasp(design = input, response = output, lower_bound=FALSE)
```

```
## The upper bounds of the range parameters are Inf Inf Inf Inf Inf Inf Inf Inf
## The initial values of range parameters are 0.001059729 0.01999988 0.01999989 0.01995154 0.01988913 0
## Start of the optimization  1  :
## The number of iterations is  28
##  The value of the  marginal posterior  function is  -292.7766
##  Optimized range parameters are 880131429831218195204024206600660042220402224240682228042862080624801
##  Optimized nugget parameter is 0
##  Convergence:  FALSE
## The initial values of range parameters are 0.6233334 310593 329576.8 754.7742 329.4202 740.7992 3462
## Start of the optimization  2  :
## The number of iterations is  67
##  The value of the  marginal posterior  function is  -127.4453
##  Optimized range parameters are 0.1950828 15754792663288 146077093533443 771.5979 41857.69 873.9928 3
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
```

```
P <- findInertInputs(m)
```

```
## The estimated normalized inverse range parameters are : 4.031267 0.00000002487248 0.000000002846522
## The inputs  2 3 5 are suspected to be inert inputs
```

Similar to the automatic relevance determination model in neural networks, e.g. MacKay (1996); Neal (1996), and in machine learning, e.g. Tipping (2001); Li et al. (2002), the function `findInertInputs` of the `RobustGaSP` package indicates that the 2nd, 3rd, and 5th inputs are suspected to be inert inputs. Figure 4 presents the plots of the borehole function when varying one input at a time. This analyzes the local sensitivity of an input when having the others fixed. Indeed, the output of the borehole function changes very little when the 2nd, 3rd, and 5th inputs vary.

Noisy outputs

Using only the influential inputs of the borehole function, we construct the GaSP emulator with a nugget based on 30 maximin LH samples through the following code:

```
m.subset <- rgasp(design = input[ ,c(1,4,6,7,8)], response = output,
                  nugget.est=TRUE)
```

```
## The upper bounds of the range parameters are 19.34479 23423.98 22990.27 107444.3 416986.6 Inf
## The initial values of range parameters are 0.3868958 468.4796 459.8055 2148.887 8339.732
## Start of the optimization  1  :
## The number of iterations is  45
##  The value of the  marginal posterior  function is  -126.1572
##  Optimized range parameters are 0.1836021 693.1441 777.1468 3167.149 33145.77
##  Optimized nugget parameter is 0.000000000000003253473
##  Convergence:  TRUE
## The initial values of range parameters are 0.2240224 271.2614 266.2389 1244.26 4828.915
## Start of the optimization  2  :
## The number of iterations is  45
##  The value of the  marginal posterior  function is  -126.1572
##  Optimized range parameters are 0.1836022 693.1457 777.1462 3167.157 33145.8
##  Optimized nugget parameter is 0.000000000000005056662
##  Convergence:  TRUE
```

```
m.subset
```

```
##
## Call:
## rgasp(design = input[, c(1, 4, 6, 7, 8)], response = output,
##     nugget.est = TRUE)
## Mean parameters:  142.1235
## Variance parameter:  76967.69
## Range parameters:  0.1836021 693.1441 777.1468 3167.149 33145.77
## Noise parameter:  0.0000000002504123
```

To compare the performance of the emulator with and without a noise term, we perform some out-of-sample testing. We build the GaSP emulator by the `RobustGaSP` package and the `DiceKriging` package using the same mean and covariance. In `RobustGaSP`, the parameters in the correlation functions are estimated by marginal posterior modes with the robust parameterisation, while in `DiceKriging`, parameters are estimated by MLE with upper and lower bounds. We first construct these four emulators with the following code

```
m.full <- rgasp(design = input, response = output)
```

```
## The upper bounds of the range parameters are 123.6899 61631867 65398881 149772 65367.81 146999 686995
## The initial values of range parameters are 2.473797 1232637 1307978 2995.441 1307.356 2939.979 13739
## Start of the optimization  1  :
## The number of iterations is  31
##  The value of the  marginal posterior  function is  -127.7353
##  Optimized range parameters are 0.1932037 61631867 65398881 709.2224 31879.72 806.2708 3478.621 33796
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
## The initial values of range parameters are 0.6233334 310593 329576.8 754.7742 329.4202 740.7992 3462
```

```
## Start of the optimization  2  :
## The number of iterations is  31
##  The value of the  marginal posterior  function is  -127.7353
##  Optimized range parameters are 0.1932036 61631867 65398881 709.2223 31879.86 806.2717 3478.621 3379
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
```

```
m.subset <- rgasp(design = input[ ,c(1,4,6,7,8)], response = output,
                  nugget.est=TRUE)
```

```
## The upper bounds of the range parameters are 19.34479 23423.98 22990.27 107444.3 416986.6 Inf
## The initial values of range parameters are 0.3868958 468.4796 459.8055 2148.887 8339.732
## Start of the optimization  1  :
## The number of iterations is  45
##  The value of the  marginal posterior  function is  -126.1572
##  Optimized range parameters are 0.1836021 693.1441 777.1468 3167.149 33145.77
##  Optimized nugget parameter is 0.00000000000003253473
##  Convergence:  TRUE
## The initial values of range parameters are 0.2240224 271.2614 266.2389 1244.26 4828.915
## Start of the optimization  2  :
## The number of iterations is  45
##  The value of the  marginal posterior  function is  -126.1572
##  Optimized range parameters are 0.1836022 693.1457 777.1462 3167.157 33145.8
##  Optimized nugget parameter is 0.00000000000005056662
##  Convergence:  TRUE
```

```
dk.full <- km(design = input, response = output)
```

```
##
## optimisation start
## ------------------
## * estimation method   : MLE
## * optimisation method : BFGS
## * analytical gradient : used
## * trend model : ~1
## * covariance model :
##   - type :  matern5_2
##   - nugget : NO
##   - parameters lower bounds :  0.0000000001 0.0000000001 0.0000000001 0.0000000001 0.0000000001 0.000
##   - parameters upper bounds :  0.1959501 97637.49 103605.2 237.2696 103.556 232.8764 1088.341 4223.8
##   - best initial criterion value(s) :  -175.6162
##
## N = 8, M = 5 machine precision = 2.22045e-16
## At X0, 0 variables are exactly at the bounds
## At iterate     0  f=       175.62  |proj g|=      0.16924
## At iterate     1  f =        172.3  |proj g|=      0.073955
## At iterate     2  f =       171.83  |proj g|=      0.065994
## At iterate     3  f =       171.66  |proj g|=       0.14412
## At iterate     4  f =        171.6  |proj g|=      0.057194
## At iterate     5  f =        171.6  |proj g|=      0.056321
## At iterate     6  f =        171.6  |proj g|=      0.051184
## At iterate     7  f =        171.6  |proj g|=      0.056248
## At iterate     8  f =        171.6  |proj g|=      0.056325
```

```
## At iterate     9  f =        171.6  |proj g|=      0.056448
## At iterate    10  f =        171.6  |proj g|=      0.056649
## At iterate    11  f =        171.6  |proj g|=      0.056975
## At iterate    12  f =        171.6  |proj g|=      0.057508
## At iterate    13  f =       171.59  |proj g|=      0.058376
## At iterate    14  f =       171.57  |proj g|=      0.059816
## At iterate    15  f =       171.51  |proj g|=      0.062152
## At iterate    16  f =       171.36  |proj g|=      0.065673
## At iterate    17  f =       171.11  |proj g|=      0.067085
## At iterate    18  f =        170.9  |proj g|=      0.060556
## At iterate    19  f =       170.87  |proj g|=       0.13976
## At iterate    20  f =       170.87  |proj g|=      0.057084
## At iterate    21  f =       170.87  |proj g|=     0.0085229
## At iterate    22  f =       170.87  |proj g|=     0.0085225
##
## iterations 22
## function evaluations 26
## segments explored during Cauchy searches 23
## BFGS updates skipped 0
## active bounds at final generalized Cauchy point 1
## norm of the final projected gradient 0.00852255
## final function value 170.867
##
## F = 170.867
## final  value 170.866677
## converged
```

```
dk.subset <- km(design = input[ ,c(1,4,6,7,8)], response = output,
                nugget.estim=TRUE)
```

```
##
## optimisation start
## ------------------
## * estimation method   : MLE
## * optimisation method : BFGS
## * analytical gradient : used
## * trend model : ~1
## * covariance model :
##   - type :  matern5_2
##   - nugget : unknown homogenous nugget effect
##   - parameters lower bounds :  0.0000000001 0.0000000001 0.0000000001 0.0000000001 0.0000000001
##   - parameters upper bounds :  0.1959501 237.2696 232.8764 1088.341 4223.802
##   - upper bound for alpha   :  1
##   - best initial criterion value(s) :  -200.1697
##
## N = 6, M = 5 machine precision = 2.22045e-16
## At X0, 0 variables are exactly at the bounds
## At iterate     0  f=       200.17  |proj g|=      0.84835
## At iterate     1  f =       175.57  |proj g|=       0.31708
## At iterate     2  f =       145.53  |proj g|=       0.15171
## ys=-2.211e+02  -gs= 1.281e+01, BFGS update SKIPPED
## At iterate     3  f =       142.16  |proj g|=      0.094111
## At iterate     4  f =       141.94  |proj g|=      0.087472
## At iterate     5  f =       141.84  |proj g|=       0.11877
```

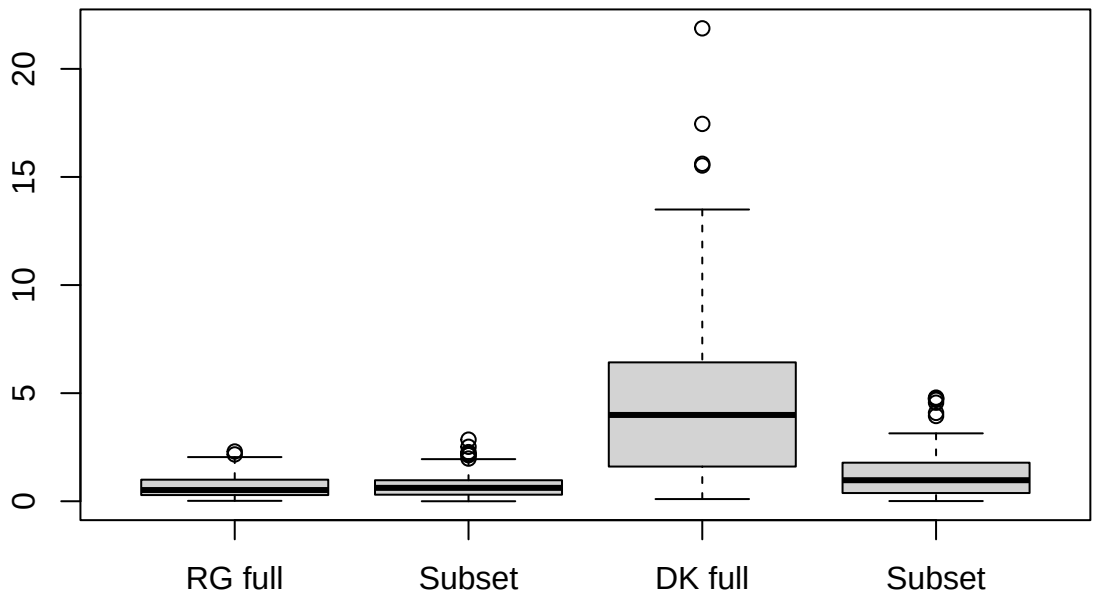
```
## At iterate     6  f =       141.83  |proj g|=      0.079792
## At iterate     7  f =       141.83  |proj g|=      0.051442
## At iterate     8  f =       141.83  |proj g|=      0.051438
## At iterate     9  f =       141.83  |proj g|=      0.051438
##
## iterations 9
## function evaluations 13
## segments explored during Cauchy searches 11
## BFGS updates skipped 1
## active bounds at final generalized Cauchy point 1
## norm of the final projected gradient 0.0514375
## final function value 141.829
##
## F = 141.829
## final  value 141.828536
## converged
```

We then compare the performance of the four different emulators at 100 random inputs for the borehole function.

```
set.seed(1)
dim_inputs <- dim(input)[2]
num_testing_input <- 100
testing_input <- matrix(runif(num_testing_input*dim_inputs),
                        num_testing_input,dim_inputs)
for(i in 1:8) {
  testing_input[,i] <- LB[i] + range[i] * testing_input[,i]
}
m.full.predict <- predict(m.full, testing_input)
m.subset.predict <- predict(m.subset, testing_input[ ,c(1,4,6,7,8)])
dk.full.predict <- predict(dk.full, newdata = testing_input,type = 'UK', checkNames = F)
dk.subset.predict <- predict(dk.subset,
                             newdata = testing_input[ ,c(1,4,6,7,8)],type = 'UK', checkNames = F)
testing_output <- matrix(0, num_testing_input, 1)
for(i in 1:num_testing_input) {
  testing_output[i] <- borehole(testing_input[i, ])
  }
m.full.error <- abs(m.full.predict$mean - testing_output)
m.subset.error <- abs(m.subset.predict$mean - testing_output)
dk.full.error <- abs(dk.full.predict$mean - testing_output)
dk.subset.error <- abs(dk.subset.predict$mean - testing_output)
```

Since the DiceKriging package seems not to have implemented a method to estimate the noise parameter, we only compare it with the nugget case. The box plot of the absolute errors of these 4 emulators (all with the same correlation and mean function) at 100 held-out points are shown in Figure 5. The performance of the RobustGaSP package based on the full set of inputs or only influential inputs with a noise is similar, and they are both better than the predictions from the DiceKriging package.

```
boxplot(
  cbind(m.full.error, m.subset.error, dk.full.error, dk.subset.error),
  names = c("RG full", "Subset", "DK full", "Subset")
  )
```



An overview of RobustGaSP

Main functions

- The mean and variance parameters are handled in a fully Bayesian way.
- The range parameters in the correlation function are estimated by numerical search for the marginal posterior modes in Equation (6). These can also be fixed, instead of estimating them. The maximum number of iterations and tolerance bounds are allowed to be chosen by users with the default setting as `max_eval=30` and `xtol_rel=1e-5`, respectively.
- The identification of inert inputs can be performed using the findInertInput function. As it only depends on the inverse range parameters through Equation (9), there is no extra computational cost in their identification (once the robust GaSP model has been built through the rgasp function). We suggest using the jointly robust prior by setting the argument prior_choice="ref_approx" in the rgasp function before calling the findInertInput function, because the penalty given by this prior is close to an L1 penalty for the logarithm of the marginal likelihood (with the choice of default prior parameters) and, hence, it can shrink the parameters for those inputs with small effect.

The optimisation algorithm

Although maximum marginal posterior mode estimation with the robust parameterisation eliminates the problems of the correlation matrix being estimated to be either $\mathbf{I}_n$ or $\mathbf{1}_n\mathbf{1}_n^T$, the correlation matrix could still be close to these singularities in some scenarios, particularly when the sample size is very large. In

such cases, we also utilize an upper bound for the range parameters γ (equivalent to a lower bound for $\beta = 1/\gamma$). The derivation of this bound is discussed in the Appendix. This bound is implemented in the `rgasp` function through the argument lower_bound=TRUE, and this is the default setting in `RobustGaSP`. **As use of the bound is a somewhat ad hoc fix for numerical problems, we encourage users to also try the analysis without the bound; this can be done by specifying `lower_bound=FALSE`.** If the answers are essentially unchanged, one has more confidence that the parameter estimates are satisfactory. Furthermore, if the purpose of the analysis is to detect inert inputs, users are also suggested to use the argument `lower_bound=FALSE` in the `rgasp` function.

Since the marginal posterior distribution could be multi-modal, the package allows for different initial values in the optimization by setting the argument `multiple_starts=TRUE` in the `rgasp` function.

- The **first default initial value** for each inverse range parameter is set to be 50 times their default lower bounds, so the starting value will not be too close to the boundary.
- The **second initial value** for each of the inverse range parameter is set to be half of the mean of the jointly robust prior. Two initial values of the nugget-variance parameter are set to be $\eta = 0.0001$ and $\eta = 0.0002$ respectively.

Examples

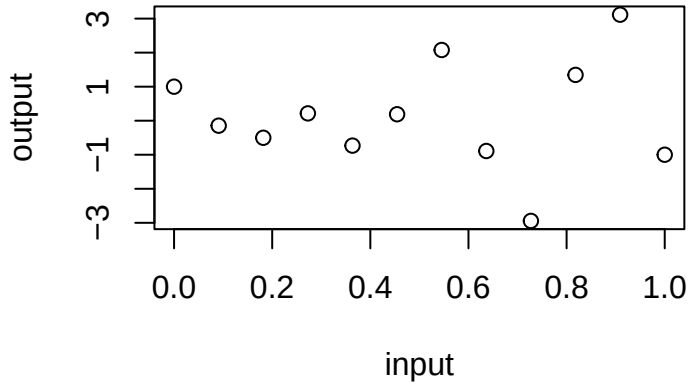
The modified sine wave function

It is expected that, for a one-dimensional function, both packages will perform well with an adequate number of design points, so we start with the function called the modified sine wave discussed in Gu (2016). It has the form

$$y = 3\sin(5\pi x) + \cos(7\pi x),$$

where $x = [0, 10]$. We first perform emulation based on 12 equally spaced design points on [0,1].

```
sinewave <- function(x) {
  3*sin(5*pi*x)*x + cos(7*pi*x)
  }
input <- as.matrix(seq(0, 1, 1/11))
output <- sinewave(input)
plot(output ~ input)
```



The GaSP model is fitted by both the RobustGaSP and DiceKriging packages, with the constant mean function.

```
m <- rgasp(design=input, response=output)
```

```
## The upper bounds of the range parameters are 72.86404
## The initial values of range parameters are 1.457281
## Start of the optimization  1  :
## The number of iterations is  7
##  The value of the  marginal posterior  function is  -19.6194
##  Optimized range parameters are 0.04072543
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
## The initial values of range parameters are 0.01388889
## Start of the optimization  2  :
## The number of iterations is  8
##  The value of the  marginal posterior  function is  -19.6194
##  Optimized range parameters are 0.04072543
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
```

```
m
```

```
##
## Call:
## rgasp(design = input, response = output)
## Mean parameters:  0.1402334
## Variance parameter:  2.603344
## Range parameters:  0.04072543
## Noise parameter:  0
```

```
dk <- km(design = input, response = output)
```

```
##
## optimisation start
## ------------------
## * estimation method   : MLE
## * optimisation method : BFGS
## * analytical gradient : used
## * trend model : ~1
## * covariance model :
##   - type :  matern5_2
##   - nugget : NO
##   - parameters lower bounds :  0.0000000001
##   - parameters upper bounds :  2
##   - best initial criterion value(s) :  -22.45764
##
## N = 1, M = 5 machine precision = 2.22045e-16
## At X0, 0 variables are exactly at the bounds
## At iterate     0  f=       22.458  |proj g|=     0.063426
## At iterate     1  f =       22.097  |proj g|=            0
##
```

```
## iterations 1
## function evaluations 2
## segments explored during Cauchy searches 1
## BFGS updates skipped 0
## active bounds at final generalized Cauchy point 1
## norm of the final projected gradient 0
## final function value 22.0969
##
## F = 22.0969
## final  value 22.096865
## converged
```

```
dk
```

```
##
## Call:
## km(design = input, response = output)
##
## Trend  coeff.:
##                Estimate
##  (Intercept)     0.1443
##
## Covar. type  : matern5_2
## Covar. coeff.:
##                 Estimate
## theta(design)     0.0000
##
## Variance estimate: 2.327824
```

A big difference between two packages is the estimated range parameter (obtained by finding `1 / m@beta_hat`) which is found to be around 0.04 in the `RobustGaSP` package, whereas it is found to be very close to zero (0) in the `DiceKriging` package (obtained from 0). To see which estimate is better, we perform prediction on 100 test points, equally spaced in [0,1].

```
testing_input <- as.matrix(seq(0, 1, 1/99))
m.predict <- predict(m, testing_input)
dk.predict <- predict(dk, testing_input, type='UK', checkNames = F)
```

```
testing_output <- sinewave(testing_input)

plot(testing_input, m.predict$mean,type='l', col='blue',
     xlab='input', ylab='output',
     ylim = c(-5,5))
polygon( c(testing_input,rev(testing_input)), c(m.predict$lower95,
                                                rev(m.predict$upper95)), col = "grey80", border = F)
lines(testing_input, testing_output)
lines(testing_input,m.predict$mean, type='l', col='blue')
lines(testing_input,dk.predict$mean, type='l', col='red')
lines(input, output, type='p', pch=19, cex=0.5)
```

The emulation results are plotted in Figure 6. Note that the red curve from the `DiceKriging` package degenerates to the fitted mean with spikes at the design points. This unsatisfying phenomenon, discussed in

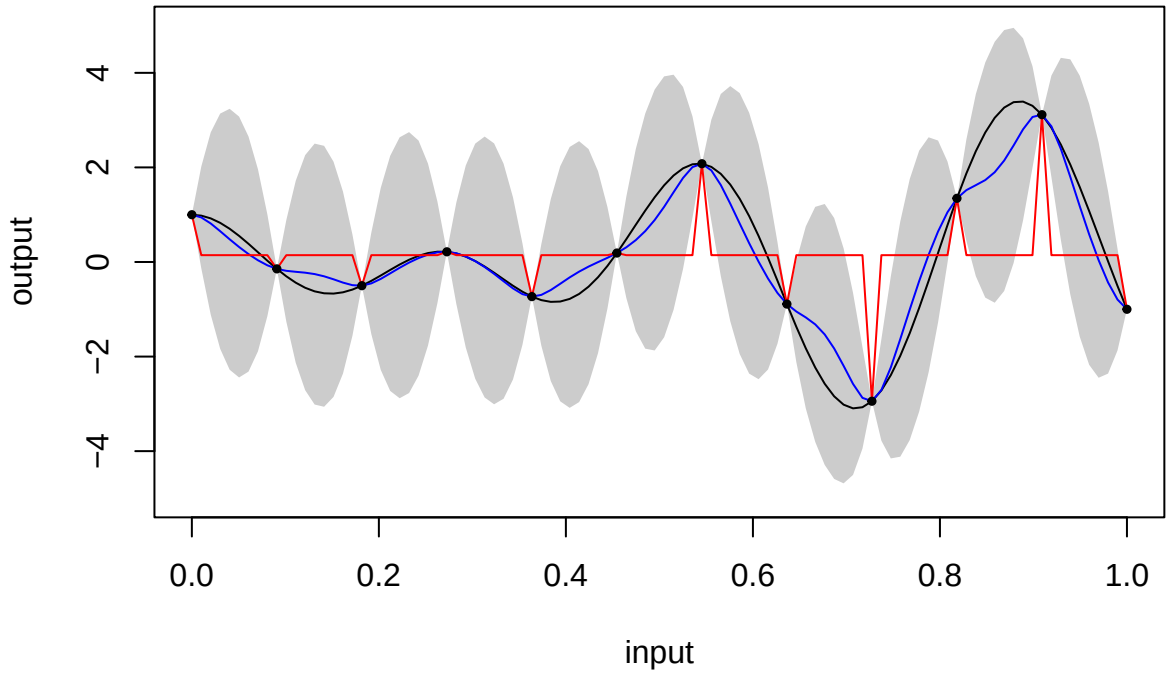
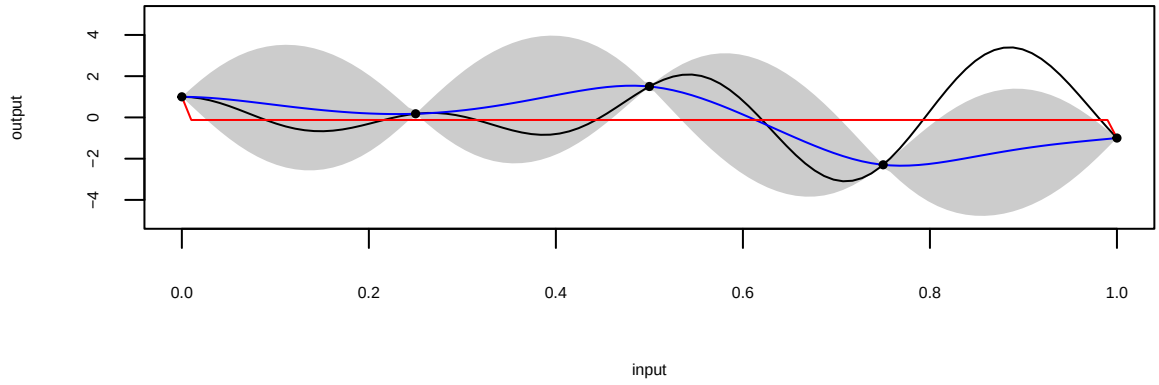
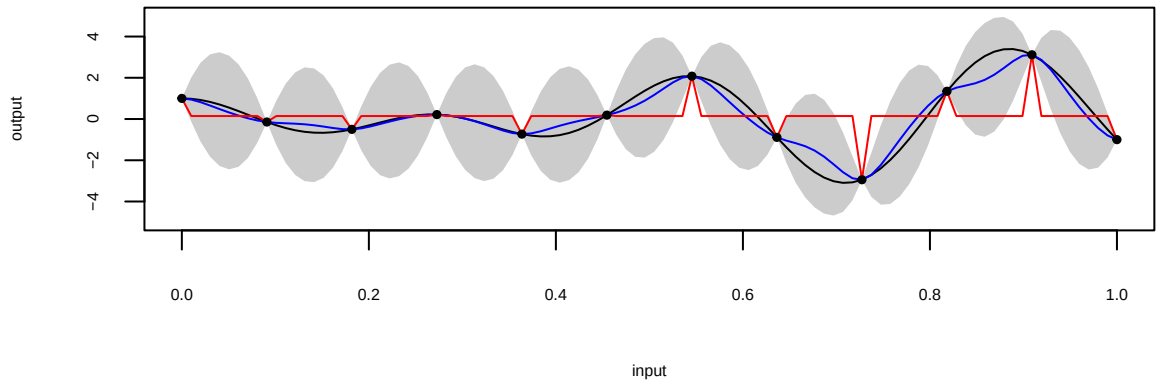


Figure 4: Figure 6

Gu et al. (2018), happens when the estimated covariance matrix is close to an identity matrix, i.e., $\hat{\mathbf{R}} \approx \mathbf{I}_n$, or equivalently $\hat{\gamma}$ tends to 0. Repeated runs of the `DiceKriging` package under different initializations yielded essentially the same results.

In their (Matern 5/2) example, changing from 5 to 12 and then to 50 observations has the following effects on the estimates:





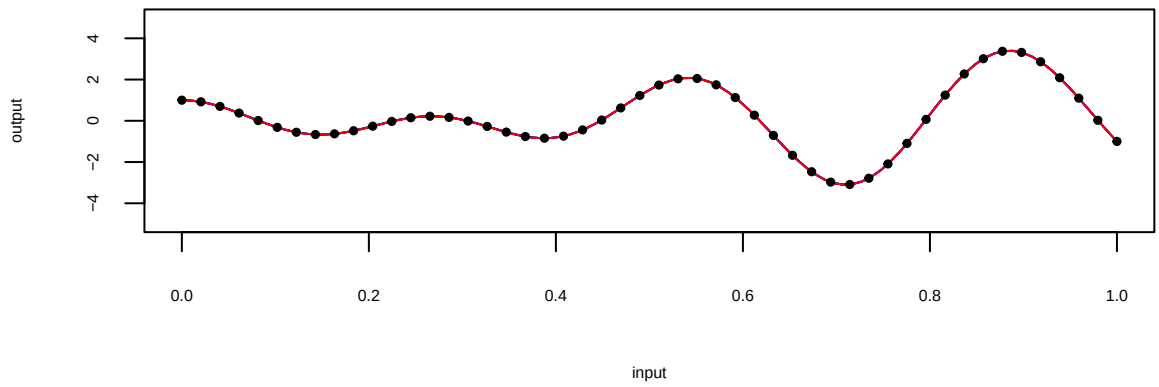


Table 1: Estimates table with 5 observations

Package	Kernel	Trend	Range	Variance
DiceKriging	Matern 5/2	-0.12	0	1.9
RobustGaSP	Matern 5/2	-0.11	0.13	2.58

Table 2: Estimates table with 12 observations

Package	Kernel	Trend	Range	Variance
DiceKriging	Matern 5/2	0.14	0	2.33
RobustGaSP	Matern 5/2	0.14	0.04	2.6

Table 3: Estimates table with 50 observations

Package	Kernel	Trend	Range	Variance
DiceKriging	Matern 5/2	-2.38	0.4	94.85
RobustGaSP	Matern 5/2	-3.25	0.48	216.15

The predictive mean from the `RobustGaSP` package is plotted as the blue curve in Figure 6 and is quite accurate as an estimate of the true function. Note, however, that the uncertainty in this prediction is quite large, as shown by the wide 95% posterior credible regions.

In this example, adding a nugget is not helpful in `DiceKriging`, as the problem is that $\hat{\mathbf{R}} \approx \mathbf{I}_n$; adding a nugget is only helpful when the correlation estimate is close to a singular matrix (i.e., $\mathbf{R} \approx \mathbf{1}_n \mathbf{1}_n^T$). However, increasing the sample size is helpful for the parameter estimation. Indeed, emulation of the modified sine wave function using 13 equally spaced design points in [0,1] was successful for one run of `DiceKriging`, as shown in the right panel of Figure 7. However, the left panel in Figure 7 gives another run of `DiceKriging` for this data, and this one converged to the problematical $\gamma \approx 0$.

Table 4: Estimates table with 13 observations (successful run)

Package	Kernel	Trend	Range	Variance
DiceKriging	Matern 5/2	0.12	0.06	1019
RobustGaSP	Matern 5/2	0.09	0.08	85201.65

Table 5: Estimates table with 13 observations (unsuccessful run)

Package	Kernel	Trend	Range	Variance
DiceKriging	Matern 5/2	0.15	0	1019
RobustGaSP	Matern 5/2	0.09	0.08	85201.65

The predictive mean from `RobustGaSP` is stable. Interestingly, the uncertainty produced by `RobustGaSP` decreased markedly with the larger number of design points.

No matter the number of observations, or whether or not 'DiceKriging' converges to the problematic $\gamma \approx 0$, the variance parameter remains the same in the repective packages: 1019 for 'DiceKriging' and 85202 for 'RobustGaSP'.

It is somewhat of a surprise that even emulation of a smooth one-dimensional function can be problematical. The difficulties with a multi-dimensional input space can be considerably greater, as indicated in the next example.

The Friedman function

Constant mean basis function

The Friedman function was introduced in Friedman (1991) and is given by

$$y = 10\sin(\pi x_1 x_2) + 20(x_3 - 0.5)^2 + 10x_4 + 5x_5,$$

, where $x_i \in [0,1]]$ for $i = 1, \ldots, 5$. 40 design points are drawn from maximin LH samples. A GaSP model is fitted using the `RobustGaSP` package and the `DiceKriging` package with the constant mean basis function (i.e., $h(\mathbf{x}) = 1$).

```
set.seed(1)
input <- maximinLHS(n=40, k=5)
num_obs <- dim(input)[1]
output <- rep(0, num_obs)
for(i in 1:num_obs) {
  output[i] <- friedman.5.data(input[i,])
  }
m <-rgasp(design=input, response=output)
```

```
## The upper bounds of the range parameters are 165.3334 166.1085 166.1872 165.9523 166.9367
## The initial values of range parameters are 3.306667 3.32217 3.323744 3.319045 3.338734
## Start of the optimization  1  :
## The number of iterations is  34
##  The value of the  marginal posterior  function is  -83.17544
##  Optimized range parameters are 2.011064 2.234651 4.720189 21.81867 39.26844
##  Optimized nugget parameter is 0
```

```
##  Convergence:  FALSE
## The initial values of range parameters are 2.196505 2.206802 2.207848 2.204727 2.217805
## Start of the optimization  2  :
## The number of iterations is  37
##  The value of the  marginal posterior  function is  -83.17544
##  Optimized range parameters are 2.011063 2.234653 4.720187 21.81867 39.26837
##  Optimized nugget parameter is 0
##  Convergence:  FALSE
```

```
dk <- km(design=input, response=output)
```

```
##
## optimisation start
## ------------------
## * estimation method   : MLE
## * optimisation method : BFGS
## * analytical gradient : used
## * trend model : ~1
## * covariance model :
##   - type :  matern5_2
##   - nugget : NO
##   - parameters lower bounds :  0.0000000001 0.0000000001 0.0000000001 0.0000000001 0.0000000001
##   - parameters upper bounds :  1.92126 1.930267 1.931182 1.928452 1.939891
##   - best initial criterion value(s) :  -92.26329
##
## N = 5, M = 5 machine precision = 2.22045e-16
## At X0, 0 variables are exactly at the bounds
## At iterate     0  f=       92.263  |proj g|=       1.4342
## At iterate     1  f =       87.666  |proj g|=         1.198
## At iterate     2  f =       84.456  |proj g|=       0.97015
## At iterate     3  f =       82.614  |proj g|=        1.2761
## At iterate     4  f =       81.899  |proj g|=       0.95394
## At iterate     5  f =       81.586  |proj g|=       0.88877
## At iterate     6  f =       81.014  |proj g|=         1.174
## At iterate     7  f =       80.692  |proj g|=       0.82438
## At iterate     8  f =       80.686  |proj g|=       0.53797
## At iterate     9  f =       80.683  |proj g|=       0.15511
## At iterate    10  f =       80.682  |proj g|=    0.00063824
## At iterate    11  f =       80.682  |proj g|=     1.663e-06
##
## iterations 11
## function evaluations 14
## segments explored during Cauchy searches 15
## BFGS updates skipped 0
## active bounds at final generalized Cauchy point 2
## norm of the final projected gradient 1.66299e-06
## final function value 80.6823
##
## F = 80.6823
## final  value 80.682319
## converged
```

Prediction on 200 test points, uniformly sampled from $[0,1]^5$, is then performed.

```
dim_inputs <- dim(input)[2]
num_testing_input <- 200
testing_input <-
  matrix(runif(num_testing_input * dim_inputs),
         num_testing_input, dim_inputs)
m.predict <- predict(m, testing_input)
dk.predict <- predict(dk, testing_input, type='UK', checkNames=F)
```

To compare the performance, we calculate the root mean square errors (RMSE) for both methods,

$$RMSE = \sqrt{\frac{\Sigma_{i=1}^{n^*}(\hat{y}(\mathbf{x}_i^*) - y(\mathbf{x}_i^*))^2}{n^*}}$$

, where $y(\mathbf{x}_i^*)$ is the i-th held-out output and $\hat{y}(\mathbf{x}_i^*)$ is the prediction for $\mathbf{x}_i^*$ by the emulator, for $i = 1, \ldots, n^*$.

```
testing_output <- matrix(NA, num_testing_input, 1)
for(i in 1:num_testing_input) {
  testing_output[i] <- friedman.5.data(testing_input[i,])
  }
m.rmse <- sqrt(mean((m.predict$mean - testing_output)^2))
m.rmse
```

```
## [1] 0.3736075
```

```
dk.rmse <- sqrt(mean((dk.predict$mean - testing_output)^2))
dk.rmse
```

```
## [1] 0.9376669
```

```
plot(testing_output ~ m.predict$mean,
     col="blue", pch=19, cex=0.25)
points(testing_output ~ dk.predict$mean,
       col="red", pch=19, cex=0.25)
abline(0,1)
```

Thus the RMSE from `RobustGaSP` is 0.28, while the RMSE from `RobustGaSP` is 0.89. The predictions versus the real outputs are plotted in Figure 8. The black circles correspond to the predictive means from the `RobustGaSP` package and are closer to the real output than the red circles produced by the `DiceKriging` package. Since both packages use the same correlation and mean function, the only difference lies in the method of parameter estimation, especially estimation of the range parameters, γ. The `RobustGaSP` package seems to do better, leading to much smaller RMSE in out-of-sample prediction.

Linear mean basis function

The Friedman function has a linear trend associated with the 4th and the 5th inputs (but not the first three) so we use this example to illustrate specifying a trend in the GaSP model. For realism (one rarely actually knows the trend for a computer model), we specify a linear trend for all variables; thus we use $\mathbf{h}(\mathbf{x}) = (1, \mathbf{x})$, where $\mathbf{x} = x_1, \ldots, x_5)$ and investigate whether or not adding this linear trend to all inputs is helpful for the prediction.

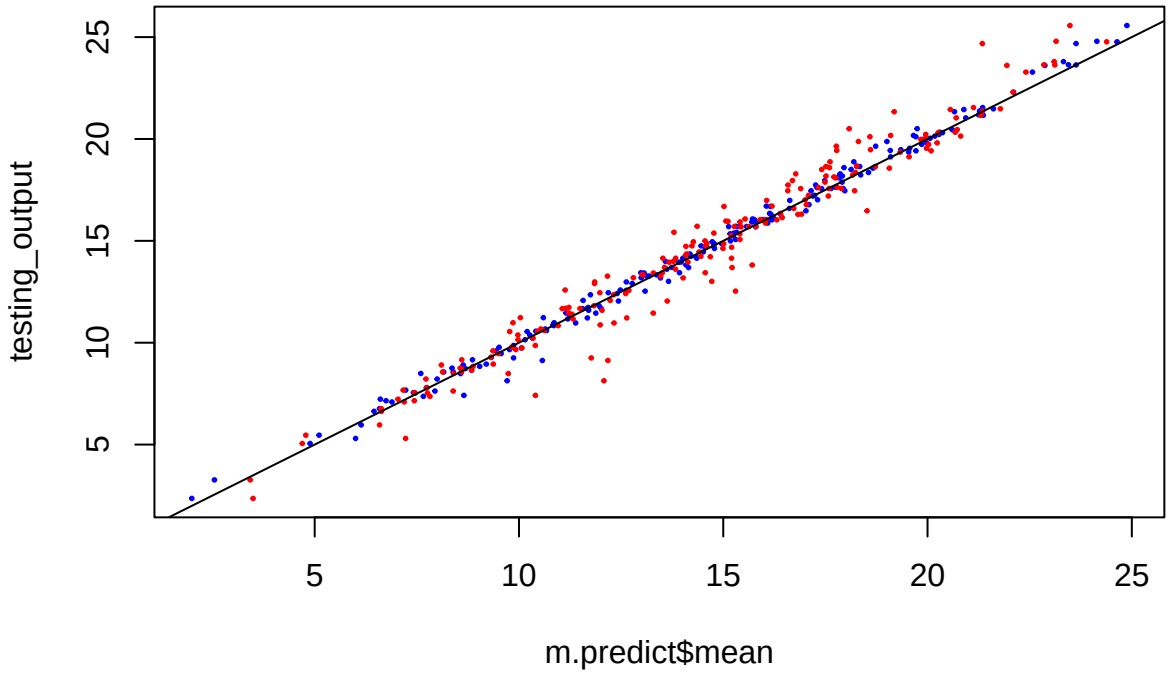


Figure 5: Figure 8

```r
set.seed(1)
colnames(input) <- c("x1", "x2", "x3", "x4", "x5")
trend.rgasp <- cbind(rep(1, num_obs), input)
m.trend <- rgasp(design=input, response=output, trend=trend.rgasp)
```

```
## The upper bounds of the range parameters are 165.3334 166.1085 166.1872 165.9523 166.9367
## The initial values of range parameters are 3.306667 3.32217 3.323744 3.319045 3.338734
## Start of the optimization  1  :
## The number of iterations is  20
##  The value of the  marginal posterior  function is  -53.49925
##  Optimized range parameters are 1.732293 1.84544 4.111523 165.9523 166.9367
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
## The initial values of range parameters are 2.196505 2.206802 2.207848 2.204727 2.217805
## Start of the optimization  2  :
## The number of iterations is  24
##  The value of the  marginal posterior  function is  -53.49925
##  Optimized range parameters are 1.732292 1.84544 4.111522 165.9523 166.9367
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
```

```r
dk.trend <- km(formula ~ x1 + x2 + x3 + x4 + x5, design=input, response=output)
```

```
##
## optimisation start
## ------------------
## * estimation method   : MLE
## * optimisation method : BFGS
## * analytical gradient : used
## * trend model : ~x1 + x2 + x3 + x4 + x5
## * covariance model :
##   - type :  matern5_2
##   - nugget : NO
##   - parameters lower bounds :  0.0000000001 0.0000000001 0.0000000001 0.0000000001 0.0000000001
##   - parameters upper bounds :  1.92126 1.930267 1.931182 1.928452 1.939891
##   - best initial criterion value(s) :  -82.15561
##
## N = 5, M = 5 machine precision = 2.22045e-16
## At X0, 0 variables are exactly at the bounds
## At iterate     0  f=       82.156  |proj g|=       1.6619
## At iterate     1  f =       79.985  |proj g|=        1.7633
## At iterate     2  f =       79.627  |proj g|=        1.4435
## At iterate     3  f =       79.432  |proj g|=        1.2359
## At iterate     4  f =       79.164  |proj g|=        1.1834
## At iterate     5  f =       79.051  |proj g|=        1.4139
## At iterate     6  f =        78.89  |proj g|=        1.7496
## At iterate     7  f =       78.603  |proj g|=        0.9932
## At iterate     8  f =       77.935  |proj g|=        1.6999
## ys=-3.837e-01  -gs= 5.127e-01, BFGS update SKIPPED
## At iterate     9  f =       77.327  |proj g|=        1.6814
## At iterate    10  f =        72.34  |proj g|=        1.6665
## At iterate    11  f =       71.455  |proj g|=        1.6204
```

```
## At iterate    12  f =       70.927  |proj g|=        1.4875
## At iterate    13  f =       69.908  |proj g|=        1.4648
## At iterate    14  f =        69.65  |proj g|=        1.4636
## At iterate    15  f =       69.598  |proj g|=        1.4005
## At iterate    16  f =       69.584  |proj g|=       0.30808
## At iterate    17  f =       69.584  |proj g|=      0.090217
## At iterate    18  f =       69.583  |proj g|=      0.035213
## At iterate    19  f =       69.583  |proj g|=      0.025549
## At iterate    20  f =       69.583  |proj g|=      0.021188
## At iterate    21  f =       69.583  |proj g|=    0.00074998
## At iterate    22  f =       69.583  |proj g|=    0.00010545
##
## iterations 22
## function evaluations 31
## segments explored during Cauchy searches 26
## BFGS updates skipped 1
## active bounds at final generalized Cauchy point 2
## norm of the final projected gradient 0.000105453
## final function value 69.5834
##
## F = 69.5834
## final  value 69.583439
## converged
```

```
colnames(testing_input) <- c("x1", "x2", "x3", "x4", "x5")
trend.test.rgasp <- cbind(rep(1, num_testing_input), testing_input)
m.trend.predict <- predict(m.trend, testing_input,
                           testing_trend=trend.test.rgasp)
dk.trend.predict <- predict(dk.trend, testing_input, type='UK')
```

```
m.trend.rmse <- sqrt(mean( (m.trend.predict$mean - testing_output)^2))
m.trend.rmse
```

```
## [1] 0.1180709
```

```
dk.trend.rmse <- sqrt(mean((dk.trend.predict$mean - testing_output)^2))
dk.trend.rmse
```

```
## [1] 1.071958
```

Adding a linear trend does improve the out-of-sample prediction accuracy of the `RobustGaSP` package; the RMSE decreases to 0.12, which is about 32% of the RMSE of the previous model with the constant mean. However, for `DiceKriging` the RMSE increases from 0.94 to 1.07, more than 9 times larger than that for the `RobustGaSP`. (**That the RMSE actually increased for `DiceKriging` is likely due to the additional difficulty of parameter estimation, since now the additional linear trend parameters needed to be estimated**; in contrast, for 'RobustGaSP', the linear trend parameters are effectively eliminated through objective Bayesian integration.) The predictions against the real output are plotted in Figure 9. The blue dots circles correspond to the predictive means from the `RobustGaSP` package, and are an excellent match to the real outputs.

```
plot(testing_output ~ m.trend.predict$mean,
     col="blue", pch=19, cex=0.25)
points(testing_output ~ dk.trend.predict$mean,
       col="red", pch=19, cex=0.25)
abline(0,1)
```

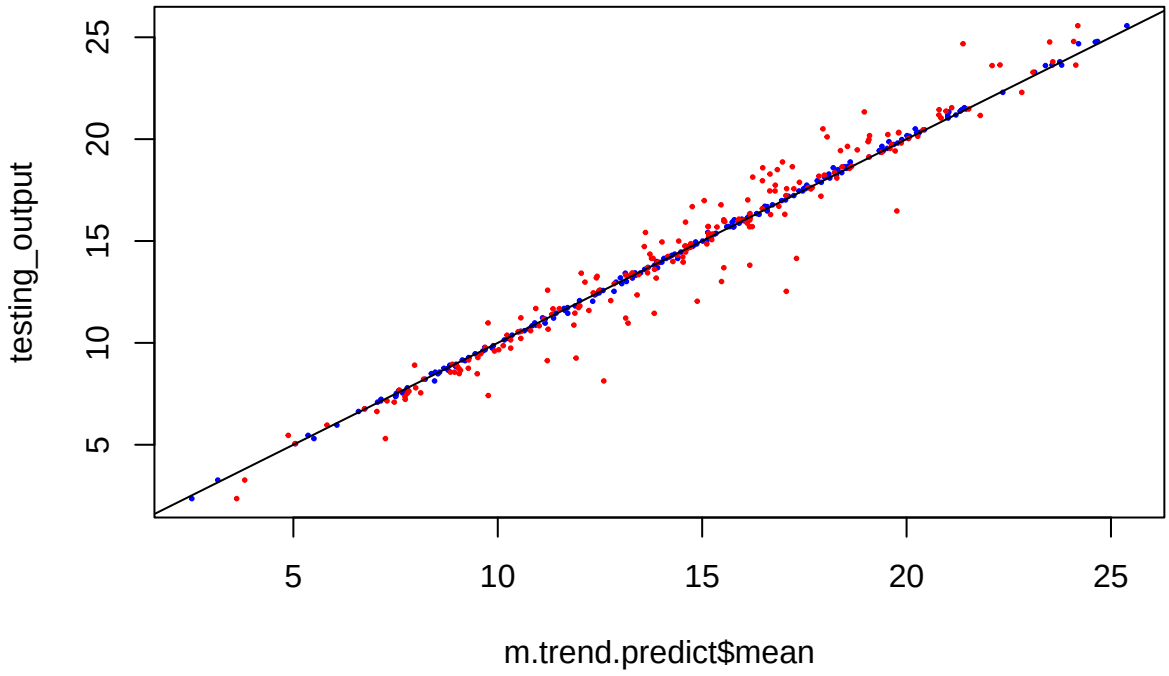


Figure 6: Figure 9